

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: КОММИВОЯЖЁР.

Студент гр. 4343

Селезнев Д. Е.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы.

Изучить методы решения задачи коммивояжёра на полном взвешенном графе. Реализовать на языке программирования Python точный метод ветвей и границ (алгоритм Литтла) и приближённый 2 – приближённый алгоритм на основе минимального остовного дерева для нахождения маршрута коммивояжёра и его стоимости по заданной матрице весов графа. Провести тестирование реализованных алгоритмов на различных входных данных и проверить корректность полученных результатов.

Задание.

Вариант №1:

МВиГ: Алгоритм Литтла. Приближённый алгоритм: 2-приближение по МОД (АДО МОД). Замечание к варианту 1. АДО МОД является 2-приближением только для евклидовой матрицы. Начинать обход МОД со стартовой вершины.

Коммивояжёр: 2-приближённый алгоритм

Разработайте программу, которая решает задачу коммивояжера при помощи 2-приближенного алгоритма. В данной постановке задачи нужно вернуться в исходную вершину после прохождения всех остальных вершин. При обходе остовного дерева (MST) необходимо идти по минимальному допустимому ребру из текущего. Каждая вершина в графе обозначается неотрицательным числом, начиная с 0, каждое ребро имеет неотрицательный вес. В графе нет рёбер из вершины в саму себя, в матрице весов на месте таких отсутствующих рёбер стоит значение -1.

В первой строке указывается начальная вершина. Далее идёт матрица весов.

В качестве выходных данных необходимо представить длину пути, полученного при помощи алгоритма. Следующей строкой необходимо представить путь, в котором перечислены вершины, по которым необходимо пройти от начальной вершины. Для приведённых в примере входных данных ответом будет

Коммивояжёр: ветви и границы

В волшебной стране Алгоритмии великий маг, Гамильтон, задумал невероятное путешествие, чтобы связать все города страны закланием процветания. Для этого ему необходимо посетить каждый город ровно один раз, создавая тропу благополучия, и вернуться обратно в столицу, используя минимум своих чародейских сил. Вашей задачей является помощь в прокладывании маршрута с помощью древнего и могущественного алгоритма ветвей и границ.

Карта дорог Алгоритмии перед Гамильтоном представляет собой полный граф, где каждый город соединён магическими порталами с каждым другим. Стоимость использования портала из города в город занимает определённое количество маны, и Гамильтон стремится минимизировать общее потребление магической энергии для закрепления проклятия.

Входные данные:

Первая строка содержит одно целое число N (N — количество городов). Города нумеруются последовательными числами от 0 до $N - 1$. Следующие N строк содержат по N чисел каждая, разделённых пробелами, формируя таким образом матрицу стоимостей M . Каждый элемент $M_{i,j}$ этой матрицы представляет собой затраты маны на перемещение из города i в город j .

Выходные данные:

Первая строка: Список из N целых чисел, разделённых пробелами, обозначающих оптимальный порядок городов в магическом маршруте Гамильтона. В начале идёт город 0, с которого начинается маршрут, затем последующие города до тех пор, пока все они не будут посещены. Вторая строка: Число, указывающее на суммарное количество израсходованной маны для завершения пути.

Теоретический минимум.

Задача коммивояжёра (TSP, Travelling Salesman Problem) — задача нахождения кратчайшего гамильтонова цикла в полном взвешенном графе.

Необходимо посетить все вершины ровно один раз, начиная и заканчивая маршрут в одной и той же вершине, минимизируя суммарный вес пройденного пути.

Граф задаётся матрицей весов, где каждый элемент $M_{i,j}$ определяет стоимость перехода из вершины i в вершину j .

В данной лабораторной работе задача решается двумя методами: точным методом ветвей и границ (алгоритм Литтла) и приближённым 2-приближённым алгоритмом на основе минимального остовного дерева (МОД).

1. Метод ветвей и границ. Алгоритм Литтла

Метод ветвей и границ позволяет находить оптимальное решение задачи коммивояжёра за счёт перебора возможных маршрутов с отсечением заведомо невыгодных вариантов.

Основная идея работы алгоритма:

- 1) Исходная матрица стоимости редуцируется:
 - из каждой строки вычитается минимальный элемент;
 - затем из каждого столбца вычитается минимальный элемент.
- 2) Сумма вычтенных значений образует нижнюю границу стоимости маршрута.
- 3) На каждом шаге выбирается нуль с максимальным штрафом, который соответствует наиболее критическому ребру для выбора в маршруте.
- 4) Задача разбивается на две подзадачи:
 - левая ветвь с включением ребра;
 - правая ветвь с его исключением.
- 5) Для каждой ветви снова вычисляется нижняя граница.
- 6) Если нижняя граница ветви больше уже найденного решения, ветвь отсекается.
- 7) Процесс продолжается до построения гамильтонова цикла.
- 8) Итоговый маршрут восстанавливается из выбранных рёбер после завершения рекурсивного перебора.

2. Приближённый алгоритм. 2-приближение на основе МОД

Данный метод позволяет получить решение быстрее, но не обязательно оптимальное.

Основная идея работы алгоритма:

- 1) По исходному графу строится минимальное остовное дерево (MST), например, алгоритмом Прима.
- 2) Выполняется обход дерева в порядке предварительного DFS (preorder), начиная со стартовой вершины.
- 3) Маршрут формируется из посещённых вершин с возвратом в исходную вершину.
- 4) Данный алгоритм является 2-приближением для метрики Евклида: длина маршрута $\leq 2 \times$ оптимальная.

Временная сложность:

- Алгоритм Литтла: в худшем случае алгоритм проверяет все возможные перестановки рёбер, что соответствует $O(n!)$, где n – число вершин графа. Практически время работы уменьшается за счёт отсечений и выбора критических рёбер. С увеличением числа вершин время растёт экспоненциально, поэтому алгоритм применим в основном для графов с малым и средним числом вершин (обычно до 15–20).

- 2-приближение по MST: построение MST – $O(n^2)$, сортировка соседей на каждом шаге DFS – $O(n \log n)$ в худшем случае для всех вершин и обход дерева – $O(n)$. Общая сложность $O(n^2)$, что делает алгоритм применимым для больших графов (сотни и тысячи вершин).

Пространственная сложность:

- Алгоритм Литтла: хранение матриц редукции на каждом шаге рекурсии – $O(n^2)$ для каждой копии, стек рекурсивных вызовов – глубина до $O(n)$, хранение текущего списка выбранных рёбер и лучшего найденного маршрута – $O(n)$ и $O(n)$ соответственно. Итоговая пространственная сложность примерно $O(n^2)$.

- 2-приближение по MST: хранение матрицы весов – $O(n^2)$, хранение MST в виде списка смежности – $O(n)$ для дерева с $n - 1$ рёбрами, стек рекурсии для DFS – глубина до $O(n)$. Итоговая пространственная сложность: $O(n^2)$.

Выполнение работы.

1. Алгоритм Литтла

Программа реализует алгоритм Литтла (метод ветвей и границ) для точного решения задачи коммивояжёра. Алгоритм гарантирует нахождение оптимального гамильтонова цикла минимальной стоимости путём систематического перебора с отсечением неперспективных ветвей.

Глобальные константы

`INF = float('inf')` – константа для представления бесконечности, используется для обозначения отсутствующих рёбер в графе и недопустимых переходов.

Вспомогательные функции

`copy_matrix(matrix: list[list[float]])` – создаёт независимую копию двумерного списка с помощью list comprehension. Используется для создания отдельных матриц для левой и правой ветвей рекурсии.

`reduce_matrix(matrix: list[list[float]])` – выполняет редукцию матрицы и возвращает стоимость редукции. Сначала для каждой строки находится минимальный элемент (не равный `INF`), который вычитается из всех элементов строки, а его значение добавляется к общей стоимости. Затем аналогичная операция выполняется для столбцов: для каждого столбца находится минимум путём перебора всех строк, и, если он положителен и не равен `INF`, вычитается из всех элементов столбца. Функция возвращает суммарную стоимость редукции, которая является нижней границей для всех решений в данной ветви.

`find_best_zero(matrix: list[list[float]])` – находит нулевой элемент матрицы с максимальным штрафом (`penalty`). Для каждого нулевого элемента в позиции (i, j) вычисляется штраф как сумма минимального элемента в i -й

строке (исключая j -й столбец) и минимального элемента в j -м столбце (исключая i -ю строку). Штраф показывает, насколько увеличится нижняя граница при исключении данного ребра. Функция возвращает позицию (i, j) с максимальным штрафом и значение этого штрафа. Если нулей нет, возвращается $(None, penalty)$.

find_chain_endpoints(edges: list[tuple[int, int]], u: int, v: int) – определяет начало и конец цепочки рёбер после добавления нового ребра (u, v) . Создаются два словаря: *next_map* для отображения «откуда $\rightarrow$ куда» и *prev_map* для обратного отображения. Временно добавляется новое ребро в оба словаря. Затем от вершины u идём назад по *prev_map* до тех пор, пока есть предшественники, находя начало цепочки (*start*). Аналогично от вершины v идём вперёд по *next_map*, находя конец цепочки (*end*). Возвращается пара $(start, end)$, которая используется для запрета обратного ребра из *end* в *start*.

creates_small_cycle(edges: list[tuple[int, int]], u: int, v: int, total_n: int) – проверяет, образует ли добавление ребра (u, v) к текущему набору рёбер преждевременный цикл длиной меньше *total_n* (общее количество городов в исходной задаче). Строится словарь *next_map* из существующих рёбер, временно добавляется новое ребро (u, v) . Начиная с вершины v , следуем по цепочке, подсчитывая длину. Если встречаем вершину u , проверяем длину цикла: если она меньше *total_n*, возвращается *True* (малый цикл обнаружен). Если цепочка обрывается до возврата в u , возвращается *False*.

remove_row_col(matrix: list[list[float]], rows: list[int], cols: list[int], row_idx: int, col_idx: int) – физически удаляет строку и столбец из матрицы. Создаётся новая матрица, в которую копируются все элементы исходной, кроме элементов с индексами *row_idx* и *col_idx*. Одновременно создаются новые списки *rows* и *cols* с помощью list comprehension, исключая элементы с соответствующими индексами. Возвращается кортеж $(new_matrix, new_rows, new_cols)$. Эта операция соответствует классическому алгоритму Литтла с физическим удалением.

forbid_edge(matrix: list[list[float]], rows: list[int], cols: list[int], from_city: int, to_city: int) – запрещает ребро между двумя городами, устанавливая соответствующий элемент матрицы в *INF*. Сначала находятся индексы *row_idx* и *col_idx* в списках *rows* и *cols*, соответствующие городам *from_city* и *to_city*, с помощью линейного поиска. Если оба индекса найдены, элемент *matrix[row_idx][col_idx]* устанавливается в *INF*.

reconstruct_path(edges: list[tuple[int, int]], n: int) – восстанавливает путь из списка рёбер. Создаётся словарь *next_map*, отображающий каждый город в следующий. Путь строится начиная с города 0, последовательно добавляя следующие города из *next_map*. Возвращается список из *n* городов, представляющий гамильтонов цикл.

calculate_cost(path: list[int], original: list[list[float]]) – вычисляет полную стоимость пути по исходной матрице. Для каждой пары соседних городов в пути (включая замыкание цикла от последнего к первому через $(i + 1) \% n$) суммируются стоимости переходов из *original* матрицы. Возвращается общая стоимость цикла.

Главная функция

little_algorithm(matrix: list[list[float]]) – основная функция, реализующая алгоритм Литтла. Сохраняет копию исходной матрицы в *original* для финального подсчёта стоимости. Создаёт редуцированную копию матрицы и вычисляет начальную нижнюю границу *initial_bound* с помощью *reduce_matrix()*. Инициализирует списки *rows* и *cols* индексами от 0 до *n*-1. Устанавливает начальные значения *best_cost* = *INF* и *best_path* = *None* для хранения оптимального решения. Определяет внутри себя вложенную рекурсивную функцию *little_recursive()* и запускает её с начальными параметрами.

little_recursive(matrix: list[list[float]], rows: list[int], cols: list[int], edges: list[tuple[int, int]], bound: float) – рекурсивная функция метода ветвей и границ. Не принимает глобальные переменные напрямую, использует *nonlocal* для доступа к *best_cost* и *best_path*. Сначала выполняется отсечение по границе:

если текущая нижняя граница $bound \geq best_cost$, ветвь отбрасывается. Если размер матрицы равен 1, это означает, что осталось последнее ребро; оно добавляется к *edges*, путь восстанавливается функцией *reconstruct_path()*, вычисляется точная стоимость через *calculate_cost()*, и, если она меньше *best_cost*, обновляются оптимальное решение.

Для ветвления функция находит нулевой элемент с максимальным штрафом с помощью *find_best_zero()*. Если нулей нет, ветвь завершается. Определяются города $u = rows[i]$ и $v = cols[j]$, соответствующие выбранному ребру. Вычисляется флаг *force_left*: если штраф больше либо равен $INF / 2$, это означает, что исключение ребра невозможно (один из минимумов бесконечен), и обрабатывается только левая ветвь.

Правая ветвь (исключение ребра): если *not force_left*, создаётся копия матрицы, элемент (i, j) устанавливается в INF (ребро запрещается), выполняется редукция для вычисления новой границы, и рекурсивно вызывается *little_recursive()* с теми же *rows*, *cols* и *edges*, но с изменённой матрицей и новой границей.

Левая ветвь (включение ребра): если добавление ребра (u, v) не создаёт малый цикл (проверка через *creates_small_cycle()*), создаётся копия матрицы, определяются концы цепочки через *find_chain_endpoints()*, физически удаляются строка i и столбец j с помощью *remove_row_col()*, запрещается обратное ребро $(end, start)$ через *forbid_edge()*, вычисляется новая граница и рекурсивно вызывается *little_recursive()* с уменьшенной матрицей, обновлёнными списками *left_rows* и *left_cols*, и расширенным списком рёбер.

Функция не возвращает значение, результат работы передаётся через изменение переменных *best_cost* и *best_path*. После завершения рекурсии *little_algorithm()* возвращает кортеж (*best_path*, $float(best_cost)$).

Пайплайн

В блоке `if __name__ == "__main__":` программа считывает количество городов n и матрицу стоимостей размером $n \times n$. Предобработка заключается

в преобразовании входных данных: значения -1 (отсутствующие рёбра) заменяются на INF для корректной работы алгоритма сравнения. Затем вызывается функция *little_algorithm(matrix)*, которая создаёт копию исходной матрицы, выполняет начальную редукцию для вычисления нижней границы и запускает рекурсивную функцию *little_recursive()* для поиска оптимального гамильтонова цикла методом ветвей и границ. В процессе рекурсии выполняются операции редукции матрицы, поиска нулевых элементов с максимальным штрафом, ветвления на включение и исключение рёбер, физического удаления строк и столбцов, запрета обратных рёбер и проверки на образование малых циклов. Постобработка сведена к восстановлению пути из списка рёбер и вычислению точной стоимости по исходной матрице. Вывод данных выполняется функцией *print*: сначала выводится оптимальный путь (последовательность городов), затем минимальная стоимость цикла.

2. 2-приближение по MST

Программа реализует 2-приближённый алгоритм решения задачи коммивояжёра на основе минимального остовного дерева (MST). Алгоритм гарантирует, что длина найденного пути не превышает удвоенной длины оптимального решения, используя построение MST и его обход в порядке DFS preorder.

Глобальные константы

$INF = \text{float}('inf')$ – константа для представления бесконечности, используется для обозначения отсутствующих рёбер в графе и недопустимых переходов.

Вспомогательные функции

read_input() – считывает входные данные из стандартного ввода. Первая строка содержит начальную вершину *start_vertex*. Последующие строки содержат матрицу весов, где отрицательные значения (обозначающие отсутствующие рёбра) заменяются на INF . Функция возвращает кортеж из начальной вершины и матрицы весов.

prim_mst(matrix: list[list[float]], n: int) – строит минимальное остовное дерево алгоритмом Прима. Инициализируется массив *min_edge*, где для каждой вершины хранится минимальное ребро, ведущее к ней из уже построенной части дерева. Начальная вершина (индекс 0) устанавливается с весом 0. На каждой итерации выбирается непосещённая вершина *u* с минимальным значением *min_edge*, помечается как посещённая, и, если у неё есть родитель, соответствующее ребро добавляется в *mst* в обе стороны (для неориентированного графа). Затем для всех непосещённых соседей *v* вершины *u* обновляются значения *min_edge*, если найдено более короткое ребро. Функция возвращает список смежности, представляющий MST.

dfs_preorder(mst: list[list[tuple[int, float]]], start: int, n: int) – выполняет обход минимального остовного дерева в порядке DFS preorder, начиная с вершины *start*. Внутри определена вложенная рекурсивная функция *dfs(v: int)*, которая помечает вершину *v* как посещённую, добавляет её в путь, сортирует соседей по весу ребра (первый приоритет) и номеру вершины (второй приоритет) с помощью *key=lambda x: (x[1], x[0])*, и рекурсивно вызывается для каждого непосещённого соседа. Сортировка соседей соответствует требованию задания. Функция возвращает список вершин в порядке их посещения.

calculate_path_length(path: list[int], matrix: list[list[float]]) – вычисляет длину пути по исходной матрице весов. Для каждой пары соседних вершин в пути суммируются веса соответствующих рёбер из *matrix*. Функция возвращает общую длину пути.

Главная функция

tsp_2approximation(start_vertex: int, matrix: list[list[float]]) – основная функция, реализующая 2-приближённый алгоритм. Сначала строится минимальное остовное дерево с помощью *prim_mst()*. Затем выполняется DFS preorder обход дерева, начиная с *start_vertex*, с помощью *dfs_preorder()*, что даёт последовательность посещения всех вершин. К полученному пути добавляется начальная вершина в конец для замыкания цикла. Длина пути

вычисляется функцией *calculate_path_length()*. Функция возвращает кортеж из длины пути и самого пути.

Пайплайн

В блоке `if __name__ == "__main__":` программа считывает начальную вершину и матрицу весов рёбер из стандартного ввода. Предобработка заключается в преобразовании входных данных: отрицательные значения (отсутствующие рёбра) заменяются на *INF* для корректной работы алгоритма сравнения. Затем вызывается функция *tsp_2approximation(start_vertex, matrix)*, которая строит минимальное остовное дерево алгоритмом Прима, начиная с вершины 0, выполняет DFS preorder обход дерева от *start_vertex* с сортировкой соседей по минимальному весу ребра и номеру вершины, и замыкает полученный путь, добавляя начальную вершину в конец. Постобработка сведена к вычислению длины пути по исходной матрице весов путём суммирования весов рёбер между последовательными вершинами. Вывод данных выполняется функцией *print*: сначала выводится длина пути с точностью до двух знаков после запятой, затем последовательность вершин пути, разделённая пробелами.

Тестирование.

Алгоритм Литтла

Для проверки корректности реализации алгоритма Литтла был разработан набор модульных и интеграционных тестов с использованием фреймворка *pytest*. Модульные тесты проверяют ключевые функции алгоритма: корректность редукции матрицы, поиск нулевого элемента с максимальным штрафом, обнаружение преждевременных циклов длиной меньше *n*, физическое удаление строк и столбцов с сохранением правильных размеров и индексов. Интеграционные тесты используют реальные данные из проверяющей системы Stepik: графы из 3 и 4 городов, а также граничный случай с 2 городами. Проверяется не только оптимальность найденного решения (соответствие ожидаемой стоимости), но и корректность структуры

пути (начало с города 0, посещение всех городов, точное соответствие ожидаемому порядку обхода).

2-приближённый алгоритм

Для проверки корректности реализации 2-приближённого алгоритма был разработан набор модульных и интеграционных тестов с использованием фреймворка *pytest*. Модульные тесты проверяют корректность построения минимального остовного дерева алгоритмом Прима: правильное количество рёбер и минимальность суммарного веса MST, а также правильность DFS preorder обхода с сортировкой соседей по минимальному весу ребра и номеру вершины. Интеграционные тесты используют реальные данные из проверяющей системы Stepik: граф из 5 городов с начальной вершиной 2, граф из 13 городов с начальной вершиной 10, а также граничные случаи с 1 и 2 городами. Проверяется точное соответствие найденного пути и его длины ожидаемым значениям, что гарантирует правильность реализации алгоритма.

Программный код реализации тестов см. в приложении А.

Результаты тестирования см. в приложении Б.

Вывод.

В ходе выполнения лабораторной работы были реализованы два алгоритма решения задачи коммивояжёра: точный алгоритм Литтла методом ветвей и границ и 2-приближённый алгоритм на основе минимального остовного дерева.

Алгоритм Литтла обеспечивает нахождение оптимального решения за счёт систематического перебора вариантов с отсечением неперспективных ветвей. Реализация включает редукцию матрицы для получения нижней границы стоимости, выбор рёбер по максимальному штрафу и физическое удаление строк и столбцов при включении ребра в маршрут. Предотвращение преждевременных циклов реализовано через отслеживание цепочек рёбер и запрет замыкающих дуг.

2-приближённый алгоритм строит решение за полиномиальное время, гарантируя, что длина найденного маршрута не превышает удвоенной длины

оптимального для симметричных графов. Алгоритм последовательно строит минимальное остовное дерево методом Прима, выполняет обход в глубину с упорядочиванием соседей по весу рёбер и номерам вершин, после чего замыкает цикл возвратом в начальную вершину.

Корректность реализации обоих алгоритмов была проверена с помощью модульных и интеграционных тестов. Модульные тесты проверяют работу отдельных функций (редукция матрицы, поиск нулевого элемента, построение MST, обход DFS), а интеграционные тесты используют реальные данные из проверяющей системы Stepik для верификации финального результата.

Таким образом, поставленная цель работы достигнута: реализованы точный и приближённый алгоритмы решения задачи коммивояжёра, изучены их практические особенности и проведена проверка корректности работы.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

tsp_little.py

```
INF = float('inf')

def copy_matrix(matrix):
    return [row[:] for row in matrix]

def reduce_matrix(matrix):
    size = len(matrix)
    cost = 0

    for i in range(size):
        row_min = min(matrix[i])
        if row_min != INF and row_min > 0:
            cost += row_min
            for j in range(size):
                if matrix[i][j] != INF:
                    matrix[i][j] -= row_min

    for j in range(size):
        col_min = INF
        for i in range(size):
            if matrix[i][j] < col_min:
                col_min = matrix[i][j]

        if col_min != INF and col_min > 0:
            cost += col_min
            for i in range(size):
                if matrix[i][j] != INF:
                    matrix[i][j] -= col_min

    return cost

def find_best_zero(matrix):
    size = len(matrix)

    best_penalty = -1
    best_pos = None

    for i in range(size):
        for j in range(size):
            if matrix[i][j] == 0:
                row_min = INF
                for k in range(size):
                    if k != j and matrix[i][k] < row_min:
                        row_min = matrix[i][k]

                col_min = INF
                for k in range(size):
                    if k != i and matrix[k][j] < col_min:
                        col_min = matrix[k][j]

                penalty = row_min + col_min
```

```

        if penalty > best_penalty:
            best_penalty = penalty
            best_pos = (i, j)

    return best_pos, best_penalty

def find_chain_endpoints(edges, u, v):
    next_map = {}
    prev_map = {}

    for a, b in edges:
        next_map[a] = b
        prev_map[b] = a

    next_map[u] = v
    prev_map[v] = u

    start = u
    while start in prev_map:
        start = prev_map[start]

    end = v
    while end in next_map:
        end = next_map[end]

    return start, end

def creates_small_cycle(edges, u, v, total_n):
    next_map = {}

    for a, b in edges:
        next_map[a] = b

    next_map[u] = v

    cur = v
    length = 1

    while cur in next_map:
        cur = next_map[cur]
        length += 1

    if cur == u:
        return length < total_n

    return False

def remove_row_col(matrix, rows, cols, row_idx, col_idx):
    new_matrix = []

    for i in range(len(matrix)):
        if i != row_idx:
            new_row = []
            for j in range(len(matrix)):
                if j != col_idx:
                    new_row.append(matrix[i][j])
            new_matrix.append(new_row)

```



```

new_rows = [rows[i] for i in range(len(rows)) if i != row_idx]
new_cols = [cols[j] for j in range(len(cols)) if j != col_idx]

return new_matrix, new_rows, new_cols

def forbid_edge(matrix, rows, cols, from_city, to_city):
    row_idx = -1
    col_idx = -1

    for i in range(len(rows)):
        if rows[i] == from_city:
            row_idx = i
            break

    for j in range(len(cols)):
        if cols[j] == to_city:
            col_idx = j
            break

    if row_idx != -1 and col_idx != -1:
        matrix[row_idx][col_idx] = INF

def reconstruct_path(edges, n):
    next_map = {}

    for u, v in edges:
        next_map[u] = v

    path = [0]
    cur = 0

    for _ in range(n - 1):
        cur = next_map[cur]
        path.append(cur)

    return path

def calculate_cost(path, original):
    total = 0
    n = len(path)

    for i in range(n):
        total += original[path[i]][path[(i + 1) % n]]

    return total

def little_algorithm(matrix):
    n = len(matrix)

    original = copy_matrix(matrix)

    reduced = copy_matrix(matrix)
    initial_bound = reduce_matrix(reduced)

    rows = list(range(n))
    cols = list(range(n))

    best_cost = INF

```

```

best_path = None

def little_recursive(matrix, rows, cols, edges, bound):
    nonlocal best_cost, best_path

    total_n = len(original)

    if bound >= best_cost:
        return

    if len(matrix) == 1:
        u = rows[0]
        v = cols[0]

        final_edges = edges + [(u, v)]

        path = reconstruct_path(final_edges, total_n)
        total_cost = calculate_cost(path, original)

        if total_cost < best_cost:
            best_cost = total_cost
            best_path = path

    return

result = find_best_zero(matrix)

if result[0] is None:
    return

(i, j), penalty = result

u = rows[i]
v = cols[j]

force_left = penalty >= INF / 2

if not force_left:
    right_matrix = copy_matrix(matrix)
    right_matrix[i][j] = INF

    right_bound = bound + reduce_matrix(right_matrix)

    little_recursive(
        right_matrix,
        rows,
        cols,
        edges,
        right_bound
    )

if not creates_small_cycle(edges, u, v, total_n):
    left_matrix = copy_matrix(matrix)

    start, end = find_chain_endpoints(edges, u, v)

    left_matrix, left_rows, left_cols = remove_row_col(
        left_matrix, rows, cols, i, j
    )

```

```

    )

    forbid_edge(left_matrix, left_rows, left_cols, end, start)

    left_bound = bound + reduce_matrix(left_matrix)

    little_recursive(
        left_matrix,
        left_rows,
        left_cols,
        edges + [(u, v)],
        left_bound
    )

    little_recursive(reduced, rows, cols, [], initial_bound)

    return best_path, float(best_cost)

def main():
    n = int(input())

    matrix = []

    for _ in range(n):
        row = list(map(int, input().split()))
        row = [INF if x == -1 else x for x in row]
        matrix.append(row)

    path, cost = little_algorithm(matrix)

    print(*path)
    print(cost)

if __name__ == "__main__":
    main()

```

tsp_2approx.py

```

import sys

INF = float('inf')

def read_input():
    lines = sys.stdin.read().strip().split('\n')
    start_vertex = int(lines[0])

    matrix = []
    for i in range(1, len(lines)):
        row = []
        for value in map(float, lines[i].split()):
            if value < 0:
                row.append(INF)
            else:
                row.append(value)
        matrix.append(row)

    return start_vertex, matrix

def prim_mst(matrix, n):

```

```

visited = [False] * n
mst = [[] for _ in range(n)]

min_edge = [(None, INF)] * n

min_edge[0] = (None, 0)

for _ in range(n):
    u = -1
    for v in range(n):
        if not visited[v] and (u == -1 or min_edge[v][1] <
min_edge[u][1]):
            u = v

    if min_edge[u][1] == INF:
        break

    visited[u] = True

    if min_edge[u][0] is not None:
        parent = min_edge[u][0]
        weight = min_edge[u][1]
        mst[parent].append((u, weight))
        mst[u].append((parent, weight))

    for v in range(n):
        if not visited[v] and matrix[u][v] < min_edge[v][1]:
            min_edge[v] = (u, matrix[u][v])

return mst

def dfs_preorder(mst, start, n):
    visited = [False] * n
    path = []

    def dfs(v: int):
        visited[v] = True
        path.append(v)

        neighbors = sorted(mst[v], key=lambda x: (x[1], x[0]))

        for neighbor, weight in neighbors:
            if not visited[neighbor]:
                dfs(neighbor)

    dfs(start)
    return path

def calculate_path_length(path, matrix):
    total = 0.0
    for i in range(len(path) - 1):
        total += matrix[path[i]][path[i + 1]]
    return total

def tsp_2approximation(start_vertex, matrix):
    n = len(matrix)

    mst = prim_mst(matrix, n)

```

```

    path = dfs_preorder(mst, start_vertex, n)

    path.append(start_vertex)

    length = calculate_path_length(path, matrix)

    return length, path

def main():
    start_vertex, matrix = read_input()
    length, path = tsp_2approximation(start_vertex, matrix)

    print(f"{length:.2f}")
    print(' '.join(map(str, path)))

if __name__ == "__main__":
    main()

```

test_tsp_little.py

```

import pytest
from tsp_little import (
    reduce_matrix,
    find_best_zero,
    creates_small_cycle,
    remove_row_col,
    little_algorithm,
    INF
)

class TestReduceMatrix:
    def test_reduce_simple_matrix(self):
        matrix = [[INF, 2, 3], [4, INF, 5], [6, 7, INF]]
        cost = reduce_matrix(matrix)
        # Редукция строк: 2+4+6=12, редукция столбцов: 1, итого: 13
        assert cost == 13
        for row in matrix:
            assert 0 in row or all(x == INF for x in row)

class TestFindBestZero:
    def test_find_in_simple_matrix(self):
        matrix = [[INF, 0, 1], [0, INF, 2], [3, 4, INF]]
        pos, penalty = find_best_zero(matrix)
        # Два нуля: (0,1) со штрафом 1+4=5, (1,0) со штрафом 2+3=5
        # Выбирается первый найденный
        assert pos == (0, 1)
        assert penalty == 5

class TestCreatesSmallCycle:
    def test_small_cycle(self):
        edges = [(0, 1), (1, 2)]
        assert creates_small_cycle(edges, 2, 0, 5)

    def test_full_cycle(self):
        edges = [(0, 1), (1, 2), (2, 3), (3, 4)]
        assert not creates_small_cycle(edges, 4, 0, 5)

class TestRemoveRowCol:

```

```

    def test_remove_from_3x3(self):
        matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
        rows = [0, 1, 2]
        cols = [0, 1, 2]
        new_matrix, new_rows, new_cols = remove_row_col(matrix, rows,
cols, 1, 1)
        assert len(new_matrix) == 2
        assert len(new_matrix[0]) == 2
        assert new_rows == [0, 2]
        assert new_cols == [0, 2]

class TestLittleAlgorithm:
    def test_sample_input_1(self):
        matrix = [
            [INF, 1, 3],
            [3, INF, 1],
            [1, 2, INF]
        ]
        path, cost = little_algorithm(matrix)

        assert path == [0, 1, 2]
        assert cost == 3.0

    def test_sample_input_2(self):
        matrix = [
            [INF, 3, 4, 1],
            [1, INF, 3, 4],
            [9, 2, INF, 4],
            [8, 9, 2, INF]
        ]
        path, cost = little_algorithm(matrix)

        assert path == [0, 3, 2, 1]
        assert cost == 6.0

class TestEdgeCases:
    def test_two_cities(self):
        matrix = [
            [INF, 10],
            [15, INF]
        ]
        path, cost = little_algorithm(matrix)
        assert path == [0, 1]
        assert cost == 25.0 # 0→1→0 = 10+15

if __name__ == "__main__":
    pytest.main([__file__, "-v"])

```

test_tsp_2approx.py

```

import pytest
from tsp_2approx import (
    prim_mst,
    dfs_preorder,
    tsp_2approximation,
    INF
)

class TestPrimMST:

```

```

def test_simple_graph(self):
    matrix = [
        [INF, 1, 2],
        [1, INF, 3],
        [2, 3, INF]
    ]
    mst = prim_mst(matrix, 3)

    total_edges = sum(len(neighbors) for neighbors in mst)
    assert total_edges == 4 # 2 ребра * 2 направления

    # Оптимальное MST: 0-1 (вес 1) + 0-2 (вес 2) = 3
    total_weight = sum(weight for neighbors in mst for _, weight in
neighbors) / 2
    assert total_weight == 3.0

class TestDFSPreorder:
    def test_sorted_neighbors(self):
        mst = [
            [(1, 10), (2, 5), (3, 7)],
            [(0, 10)],
            [(0, 5)],
            [(0, 7)]
        ]
        path = dfs_preorder(mst, 0, 4)
        # После 0 должна идти вершина с минимальным весом (2 с весом 5)
        assert path[0] == 0
        assert path[1] == 2

    def test_sorted_by_vertex_number(self):
        mst = [
            [(3, 5), (1, 5), (2, 5)],
            [(0, 5)],
            [(0, 5)],
            [(0, 5)]
        ]
        path = dfs_preorder(mst, 0, 4)
        # При равных весах должна быть сортировка по номеру
        assert path[0] == 0
        assert path[1] == 1 # Минимальный номер при равных весах

class TestSolveTSP:
    def test_sample_input_1(self):
        matrix = [
            [INF, 18.97, 22.36, 19.42, 3.61],
            [18.97, INF, 35.61, 38.01, 17.0],
            [22.36, 35.61, INF, 16.28, 21.19],
            [19.42, 38.01, 16.28, INF, 21.02],
            [3.61, 17.0, 21.19, 21.02, INF]
        ]
        length, path = tsp_2approximation(2, matrix)

        assert path == [2, 3, 0, 4, 1, 2]
        assert abs(length - 91.92) < 0.01

    def test_sample_input_2(self):
        matrix = [

```

```

        [INF, 9.22, 23.32, 8.49, 16.76, 30.0, 26.02, 13.89, 3.61,
13.15, 21.4, 8.06, 4.12],
        [9.22, INF, 32.2, 17.69, 25.5, 39.2, 33.38, 13.93, 12.73,
21.02, 30.48, 5.1, 11.66],
        [23.32, 32.2, INF, 15.23, 15.52, 12.17, 25.24, 27.29, 19.72,
12.21, 3.16, 29.0, 20.62],
        [8.49, 17.69, 15.23, INF, 10.05, 21.63, 21.19, 18.03, 5.0,
8.06, 13.04, 15.65, 7.28],
        [16.76, 25.5, 15.52, 10.05, INF, 15.26, 11.66, 28.07, 14.14,
16.12, 12.37, 24.74, 17.03],
        [30.0, 39.2, 12.17, 21.63, 15.26, INF, 19.1, 37.64, 26.63,
22.56, 11.05, 37.22, 28.65],
        [26.02, 33.38, 25.24, 21.19, 11.66, 19.1, INF, 38.83, 24.33,
27.78, 22.2, 34.0, 27.46],
        [13.89, 13.93, 27.29, 18.03, 28.07, 37.64, 38.83, INF, 14.56,
15.23, 26.93, 8.94, 11.4],
        [3.61, 12.73, 19.72, 5.0, 14.14, 26.63, 24.33, 14.56, INF,
10.0, 17.8, 10.77, 3.16],
        [13.15, 21.02, 12.21, 8.06, 16.12, 22.56, 27.78, 15.23,
10.0, INF, 11.7, 17.2, 9.49],
        [21.4, 30.48, 3.16, 13.04, 12.37, 11.05, 22.2, 26.93, 17.8,
11.7, INF, 27.66, 19.1],
        [8.06, 5.1, 29.0, 15.65, 24.74, 37.22, 34.0, 8.94, 10.77,
17.2, 27.66, INF, 8.6],
        [4.12, 11.66, 20.62, 7.28, 17.03, 28.65, 27.46, 11.4, 3.16,
9.49, 19.1, 8.6, INF]
    ]
    length, path = tsp_2approximation(10, matrix)

    assert path == [10, 2, 5, 9, 3, 8, 12, 0, 11, 1, 7, 4, 6, 10]
    assert abs(length - 147.25) < 0.01

class TestEdgeCases:
    def test_two_cities(self):
        matrix = [
            [INF, 10],
            [10, INF]
        ]
        length, path = tsp_2approximation(0, matrix)
        assert len(path) == 3
        assert length == 20

    def test_single_city(self):
        matrix = [[INF]]
        length, path = tsp_2approximation(0, matrix)
        assert len(path) == 2
        assert path == [0, 0]

if __name__ == "__main__":
    pytest.main([__file__, "-v"])

```


ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ

Таблица 1 - Результаты тестирования алгоритма Литтла

№ п/п	Входные данные	Выходные данные	Комментарии
1.	reduce_matrix(matrix)	cost = 13	Корректная работа редукции матрицы
2.	find_best_zero(matrix)	pos = (0, 1), penalty = 5	Корректная работа выбора нуля с максимальным штрафом
3.	creates_small_cycle() с ребрами [(0,1), (1,2)] и добавлением (2, 0)	True	Проверка обнаружения преждевременного цикла
4.	creates_small_cycle() с ребрами [(0,1), (1,2), (2,3), (3,4)] и добавлением (4, 0)	False	Проверка допустимости полного цикла
5.	remove_row_col() удаления строки 1, столбца 1	Матрица 2 × 2, new_rows = [0, 2], new_cols = [0, 2]	Корректная работа физического удаления строк и столбцов
6.	little_algorithm() для графа из 3 городов	path = [0, 1, 2], cost = 3.0	Корректная работа алгоритма на графе из 3 городов
7.	little_algorithm() для графа из 4 городов	path = [0, 3, 2, 1], cost = 6.0	Корректная работа алгоритма на графе из 4 городов
8.	little_algorithm() для графа из 2 городов	path = [0, 1], cost = 25.0	Граничный случай: минимальное количество городов

Таблица 2 - Результаты тестирования 2-приближенного алгоритма

№ п/п	Входные данные	Выходные данные	Комментарии
1.	prim_mst()	total_edges = 4, total_weight = 3.0	Корректная работа построения MST
2.	dfs_preorder()	path[1] = 2	Проверка сортировки соседей по весу (минимальный вес первым)
3.	dfs_preorder() с соседями одинакового веса	path[1] = 1	Проверка вторичной сортировки по номеру вершины
4.	tsp_2approximation() для графа из 5 городов, старт из вершины 2	path = [2, 3, 0, 4, 1, 2], length = 91.92	Корректная работа алгоритма на графе из 5 городов
5.	tsp_2approximation() для графа из 13 городов, старт из вершины 10	path = [10, 2, 5, 9, 3, 8, 12, 0, 11, 1, 7, 4, 6, 10], length = 147.25	Корректная работа физического удаления строк и столбцов
6.	tsp_2approximation() для графа из 2 городов	len(path) = 3, length = 20	Граничный случай: минимальное количество городов

7.	tsp_2approximation() для графа из 1 города	path = [0, 0], len(path) = 2	Граничный случай: один город
----	--	---------------------------------	------------------------------